- "For any large enough input". This is a crucial piece: we are interested in how our functions behave when `n` gets (very) large and we don't care if for small values of `n` the inequalities don't hold. Think, for example, to the functions `f(x) = e^x` and `g(x) = e * x`. `f(x) < g(x)` when `x` is smaller than `e`, but for larger values `f(x)` grows much faster.
- Constant factors don't matter: `O(n) = O(3 * n) = O(100 * n)`. You can prove it by choosing the right values for the constant in the inequality above.
- Some problems require constant time: think about summing the first n integers. With the naive algorithm, you would need `n-1` sums, but with Gauss' method, you only need one sum, one multiplication and one division, independently on the output.

  To sum this up using a formula, we can write: `O(c) = O(1)`, for any positive constant `c`.

  `O(1)` denotes a constant running time: in other words, one that does not depend on the input size.
- When summing big-O expressions, the bigger wins: `O(f(n) + g(n)) = O(f(n))` if `g(n) = O(f(n))`. So, if two algorithms are run in sequence, the total running time is dominated by the slowest.
- `O(f(n) * g(n))` can't be simplified, unless either function is constant.

Often when we give an algorithm run-time using big-O notation, we imply that the bound is both lower and upper - unless we explicitly say